CSC211 Data Structures and Algorihms Tutorial 1

11/12 February 2026

Programs can be classified in terms of the time they take, depending on the size their input n. For, example if one sorts n numbers, the timing of the sorting is given in terms of this n and not in terms of the actual instructions that are executed to do the sorting. Timing may be expressed as $T(n)$, the total time taken, e.g., $T(n) = 5n^2+3n+100$. When the size of the input n gets very big, the value of $T(n)$ becomes so dominated by the n^2 term, that nothing else really matters. Then, without now, defining it clearly, Sedgewick expresses the value of $T(n)$ *assymptotically* as $T(n) \sim O(n^2)$. Knuth in 1973, and Aho, Hopcroft and Ullman (AHU) in 1974, and Cormen, Leiserson, Rivest and Stein (CLRS) in 2022 write $T(n) = O(n^2)$.

Sometimes we can easily see how to express the timing $T(n)$ in terms of $T(n)$, e.g., $T(n) = 2T(n) + cn$. We can show that this is $O(n \log(n)$

1. Given the greatest common divisor algorithm for two numbers $m, n \in \mathbb{N}_0$ as

```
count = 0
def gcd(m,n):
    global count
    if n == 0:
        return m
    else:
        print(f"{m} {n}")
        return gcd(n, m%n)

def main():
```

   1.1. Supposing the print statement **print(f"{m} {n}")** is inserted directly after the **else**, what will be printed when **gcd(3∗7,51)** is called?

   1.2. Write the function iteratively.

   1.3. What is the value of **count** directly after a call to **gcd(3∗7,51)**?

2. Consider a program with two nested **for** loops, one inside the other. The one has the **range(n)**, and the other has the **range(m)**.

   2.1. What is $T(n)$?

   2.2. What is $(O(f(n, m))$?

   2.3. Does their order matter?

3. Consider the Towers of Hanoi problem.

3.1. What is T(n)?

3.2. What is its assymptotic size in terms of $O(f(n))$?

4. Consider the program below.

```
n = 2
d = [0]*n
count = 0
def inc (d, i):
    global count
    count += 1
    print(f"{count = }")
    n = len(d)
    if i < n:
        if d[i] < n -1:
            d[i] += 1
            return 0
        else:
            d[i] = 0
            return inc(d, i + 1)
    else:
            return 1

def counter():
    global count, d
    count = 0
    result = 0
    while result != 1:
        n = len(d)
        print ("\n inc: n = %d, d = %s" % (n, d))
        result = inc (d, 0)
    print(f"{count = }")
counter()
```

4.1. What value of **count** will be printed?

4.2. what happens to **count** when **n = 3** ?

4.3. Can you express the answer as a functionof n.

1.1)

21, 51
51, 21
21, 9
9, 3

3, 0

51 % 21 = 51
51 % 21 = 9
21 % 9 = 3
9 % 3 = 0

n == 0 return

1.2) while n != 0:

m = n
n = m % n

return m

1.3) O

---

2) $T(n) = n \times m$

2.2) $O(n \cdot m)$

2.3) Yes, anything in the inner loop is multiplied $n \times m$, so where expressions lie is important. $m \times n = m \times n$ but loops are different.

---

3) $T(n) = 2^n - 1$ $\quad (2T(n-1) + 1)$

3.2) $O(2^n)$

4) 7

4.1) 40

4.3) n^n